

□ Exercice 1 : Déclarations

- Écrire les instructions permettant de :
 - Déclarer une variable `n` de type entier.
 - Déclarer une variable `x` de type flottant initialisée à 1.
 - Déclarer une variable `test` de type booléen, quelle bibliothèque (ou en-tête) est nécessaire ?
 - Déclarer un tableau de 5 entiers initialisés aux valeurs {1, 4, 9, 16, 25}.
 - Déclarer une chaîne de caractères initialisée à « *Trop bien le langage C* ».
 - Déclarer une variable de type `char` contenant le caractère '\$'.
- Écrire les signatures des fonctions suivantes :
 - `divisible_par` qui prend en argument deux entiers n et p et renvoie un booléen.
 - `somme` qui prend en argument un tableau de flottants et un entier et renvoie un flottant.
 - `carre` qui prend en argument un entier et renvoie un entier.
 - `affiche` qui prend en argument un booléen et ne renvoie rien.

□ Exercice 2 : Boucles for

Indiquer si les boucles `for` suivantes sont correctes ou non et si elles le sont, donner les valeurs prises par la variable d'itération.

- `for (int m = 1; m < 10; m=m+1)`
- `for (int i = 12; i < 10; i++)`
- `for (int k = 0; k < 10; i--)`
- `for (j = 0; j < 10; j++)`
- `for (int i = 42; i > 21; i--)`

□ Exercice 3 : Comportements indéfinis

- Rappeler ce qu'est un *comportement indéfini* en C.
- Donner au moins un exemple de programme ayant un comportement indéfini.
- On considère la fonction suivante en C :

```
1  int signe(int n)
2  {
3      if (n>0)
4      {
5          return 1;
6      }
7      if (n<0)
8      {
9          return -1;
10     }
11 }
```

et on précise que le standard du langage C indique que :

« *If control reaches the closing curly brace (}) of a non-void function without evaluating a return statement, using the return value of the function call is undefined behavior.* »

En déduire un cas où l'utilisation de cette fonction produit un comportement indéfini et écrire une fonction `main` appelant la fonction `signe` et produisant un comportement indéfini.

Remarque : lors de la compilation, on obtient l'avertissement suivant : « *control reaches end of non-void function* ».

□ Exercice 4 : Opérations sur les types de base

- On considère le programme suivant :

```

1  #include <stdio.h>
2
3  int main()
4  {
5      float r = 5 % 3;
6      printf("Résultat = %f\n", r);
7  }

```

- Quel est le résultat produit ? Expliquer.
 - Que se passe-t-il si on change la ligne 5 en `float r = 5.0 % 3.0;` ?
- Écrire une expression booléenne valant vrai si `a` est égal à `b+c` ou si `c` est non nul et que `a` est divisible par `c`. L'évaluation de cette expression génère-t-elle une erreur si `c` est nul ?
 - On suppose déjà déclarée la variable `char c = 'A';`. Quel est le résultat de l'instruction suivante : `printf("%c \n", c+1);`

□ Exercice 5 : Portée

On considère le programme C suivant :

```

1  #include <stdio.h>
2
3  const float pi = 3.1415;
4  int k = 1;
5
6  int main() {
7      float s = 0;
8      int k = 1;
9      while (pi * pi / 6 - s > 0.25) {
10         float v;
11         v = 1.0 / (k * k);
12         s += v;
13         k = k + 1;
14     }
15     return 0;
16 }

```

- Pour chacune des variables du programme, indiquer si elle est globale ou locale et donner sa portée.
 - Déterminer la valeur de chacune des variables existantes juste avant l'instruction `return` de la ligne 15.
- 🎯 On peut utiliser une calculatrice !

□ Exercice 6 : Conversion

Déterminer le type et la valeur des expressions suivantes. Indiquer lorsqu'une conversion implicite ou explicite a eu lieu.

- `!(5<7)`
- `3 + 0.14`
- `(int) 7.5 + (int) 12.3`
- `7.0 / 2`
- `(true || false) && (false || true)`
- `(int) 19.6 % 4`

□ Exercice 7 : Analyser un programme

On considère le programme suivant :

```
1  #include <stdio.h>
2
3  void echange(int a, int b)
4  {
5      int temp = a;
6      a = b;
7      b = temp;
8  }
9
10 int main()
11 {
12     int a = 12;
13     int b = 50;
14     echange(a, b);
15     printf("a = %d\n", a);
16     printf("b = %d\n", b);
17     return 0;
18 }
```

1. Quel sera le résultat de l'exécution de ce programme ? Pourquoi ?
2. Quel sera l'affichage produit si on déplace l'affichage des variables `a` et `b` dans la fonction `echange` ? Pourquoi ?

□ **Exercice 8 :** *Programmes à commenter*

Commenter les programmes suivants en précisant les résultats obtenus (si le programme compile) ou les erreurs ou avertissements éventuels et les comportements indéfinis.

- Programme A :

```
1  #include <stdio.h>
2
3  int main()
4  {
5
6      int tab[5] = {42};
7      for (int i = 0; i < 5; i++)
8      {
9          printf("%d \n", tab[i]);
10     }
11 }
```

- Programme B :

```
1  #include <stdio.h>
2  int main()
3  {
4      int tab[5] = {42};
5      int i = 0;
6      while (i < 6)
7      {
8          printf("%d \n", tab[i]);
9          i = i + 1;
10     }
11 }
```

- Programme C :

```
1  #include <stdio.h>
2
3  int main()
4  {
5      float s = 0;
6      for (int i = 1; i < 10000; i++)
7      {
8          s = s + 1 / i;
9      }
10     printf("somme =%f\n", s);
11 }
```

- Programme D :

```
1  #include <stdio.h>
2
3  int fonction(int a)
4  {
5      a = a + 1;
6      printf("Valeur de a = %d\n", a);
7  }
8
9  int main()
10 {
11     int c = 2;
12     int d = fonction(c);
13     printf("valeur de c = %d \n", c);
14     printf("valeur de d = %d \n", d);
15 }
```

□ Exercice 9 : À propos de l'opérateur ++

1. Rappeler le type auquel il s'applique et le rôle de l'opérateur ++.
2. On suppose déjà déclarée une variable `int a = 42;`.
Que vaut `b` si on a écrit `int b = a++ + a;`?
Détaillez votre raisonnement.

□ Exercice 10 : Chasse aux bugs

Dans le programme suivant, repérer les 4 anomalies (erreurs de syntaxe, avertissements du compilateur ou comportements indéfinis) et expliquer comment les corriger :

```
1  #include <stdio.h>
2
3  int main() {
4      int total;
5      int tab[3] = {10, 20, 30};
6      for (int i = 0; i <= 3; i = i + 1) {
7          total = total + tab[i];
8      }
9      float moyenne = total / 3;
10     printf("Moyenne = %d\n", moyenne);
11     return 0;
12 }
```